Java Design Patterns - Solutions

Name : Guneet Kaur Juneja

Exercise 1: Singleton Pattern

Scenario: Logging utility class with only one instance throughout the application

Logger.java

```
public class Logger {
    private static Logger instance;

    private Logger() {
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}
```

SingletonTest.java

```
public class SingletonTest {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("First message");
        logger2.log("Second message");

        System.out.println("Same instance: " +
            (logger1 == logger2));
    }
}
```

Output:

```
[LOG] First message [LOG] Second message Same instance: true
```

Exercise 2: Factory Method Pattern

Scenario: Document management system supporting Word, PDF, and Excel documents

Document.java (Interface)

```
public interface Document {
    void open();
    void save();
    void close();
}
```

PDFDocument.java

```
public class PDFDocument implements Document {
    public void open() {
        System.out.println("Opening PDF document...");
    }

    public void save() {
        System.out.println("Saving PDF document...");
    }

    public void close() {
        System.out.println("Closing PDF document...");
    }
}
```

WordDocument.java

```
public class WordDocument implements Document {
    public void open() {
        System.out.println("Opening Word document...");
    }

    public void save() {
        System.out.println("Saving Word document...");
    }

    public void close() {
        System.out.println("Closing Word document...");
    }
}
```

DocumentFactory.java (Abstract)

```
public abstract class DocumentFactory {
    public abstract Document createDocument();

    public void openDocument() {
        Document doc = createDocument();
        doc.open();
    }
}
```

PDFDocumentFactory.java

```
public class PDFDocumentFactory extends DocumentFactory {
    public Document createDocument() {
        return new PDFDocument();
    }
}
```

FactoryTest.java

```
public class FactoryTest {
    public static void main(String[] args) {
        DocumentFactory pdfFactory = new PDFDocumentFactory();
        DocumentFactory wordFactory = new WordDocumentFactory();

        pdfFactory.openDocument();
        wordFactory.openDocument();
    }
}
```

Output:

```
Opening PDF document... Opening Word document...
```

Exercise 3: Builder Pattern

Scenario: Creating complex Computer objects with multiple optional parts

Computer.java

```
public class Computer {
    private String cpu;
    private String ram;
    private String storage;
    private String gpu;

    private Computer(ComputerBuilder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
        this.gpu = builder.gpu;
    }

    public void displayConfig() {
        System.out.println("CPU: " + cpu);
        System.out.println("RAM: " + ram);
        System.out.println("Storage: " + storage);
        System.out.println("GPU: " + (gpu != null ? gpu : "None"));
    }

    public static class ComputerBuilder {
        private String cpu;
        private String ram;
        private String storage;
        private String gpu;

        public ComputerBuilder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public ComputerBuilder setRAM(String ram) {
            this.ram = ram;
            return this;
        }

        public ComputerBuilder setStorage(String storage) {
            this.storage = storage;
            return this;
        }

        public ComputerBuilder setGPU(String gpu) {
            this.gpu = gpu;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}
```

BuilderTest.java

```
public class BuilderTest {
    public static void main(String[] args) {
        Computer pc1 = new Computer.ComputerBuilder()
            .setCPU("Intel i9")
            .setRAM("32GB")
            .setStorage("1TB SSD")
            .setGPU("RTX 4090")
            .build();

        Computer pc2 = new Computer.ComputerBuilder()
            .setCPU("AMD Ryzen 5")
            .setRAM("16GB")
            .setStorage("512GB SSD")
            .build();

        System.out.println("Configuration 1:");
        pc1.displayConfig();
        System.out.println();
        pc2.displayConfig();
    }
}
```

Output:

```
Configuration 1: CPU: Intel i9 RAM: 32GB Storage: 1TB SSD GPU: RTX 4090
Configuration 2: CPU: AMD Ryzen 5 RAM: 16GB Storage: 512GB SSD GPU: None
```

Exercise 4: Adapter Pattern

Scenario: Payment processing with multiple third-party payment gateways

PaymentProcessor.java (Target)

```
public interface PaymentProcessor {
    void processPayment(double amount);
}
```

PayPalGateway.java (Adaptee)

```
public class PayPalGateway {
    public void sendPayment(double amount) {
        System.out.println("Processing via PayPal: $" + amount);
    }
}
```

StripeGateway.java (Adaptee)

```
public class StripeGateway {
    public void charge(double amount) {
        System.out.println("Charging via Stripe: $" + amount);
    }
}
```

PayPalAdapter.java

```
public class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway gateway = new PayPalGateway();

    public void processPayment(double amount) {
        gateway.sendPayment(amount);
    }
}
```

StripeAdapter.java

```
public class StripeAdapter implements PaymentProcessor {
    private StripeGateway gateway = new StripeGateway();

    public void processPayment(double amount) {
        gateway.charge(amount);
    }
}
```

AdapterTest.java

```
public class AdapterTest {
    public static void main(String[] args) {
        PaymentProcessor paypal = new PayPalAdapter();
        PaymentProcessor stripe = new StripeAdapter();

        paypal.processPayment(50.00);
```

```
        stripe.processPayment(75.50);
    }
}
```

Output:

```
Processing via PayPal: $50.0 Charging via Stripe: $75.5
```

Exercise 5: Decorator Pattern

Scenario: Notification system with dynamic channel addition capability

Notifier.java (Interface)

```
public interface Notifier {
    void send(String message);
}
```

EmailNotifier.java

```
public class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}
```

NotifierDecorator.java (Abstract)

```
public abstract class NotifierDecorator implements Notifier {
    protected Notifier wrappedNotifier;

    public NotifierDecorator(Notifier notifier) {
        this.wrappedNotifier = notifier;
    }
}
```

SMSDecorator.java

```
public class SMSDecorator extends NotifierDecorator {
    public SMSDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        wrappedNotifier.send(message);
        System.out.println("SMS: " + message);
    }
}
```

SlackDecorator.java

```
public class SlackDecorator extends NotifierDecorator {
    public SlackDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        wrappedNotifier.send(message);
        System.out.println("Slack: " + message);
    }
}
```

DecoratorTest.java

```
public class DecoratorTest {
    public static void main(String[] args) {
        Notifier notifier = new EmailNotifier();
        notifier = new SMSDecorator(notifier);
        notifier = new SlackDecorator(notifier);

        notifier.send("Meeting at 3 PM");
    }
}
```

Output:

```
Email: Meeting at 3 PM SMS: Meeting at 3 PM Slack: Meeting at 3 PM
```

Exercise 6: Proxy Pattern

Scenario: Image viewer with lazy loading and caching from remote server

Image.java (Subject Interface)

```
public interface Image {
    void display();
}
```

RealImage.java

```
public class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadFromServer();
    }

    private void loadFromServer() {
        System.out.println("Loading image from server: " + filename);
    }

    public void display() {
        System.out.println("Displaying: " + filename);
    }
}
```

ProxyImage.java

```
public class ProxyImage implements Image {
    private String filename;
    private RealImage realImage;

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename);
        }
        realImage.display();
    }
}
```

ProxyTest.java

```
public class ProxyTest {
    public static void main(String[] args) {
        Image image = new ProxyImage("photo.jpg");
        System.out.println("Image object created");
        image.display();
        image.display();
    }
```

```
}
```

Output:

```
Image object created Loading image from server: photo.jpg Displaying: photo.jpg
Displaying: photo.jpg
```

Exercise 7: Observer Pattern

Scenario: Stock market monitoring with multiple client applications

Stock.java (Subject Interface)

```
public interface Stock {
    void registerObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}
```

Observer.java (Observer Interface)

```
public interface Observer {
    void update(double price);
}
```

MobileApp.java

```
public class MobileApp implements Observer {
    public void update(double price) {
        System.out.println("Mobile App - Stock price: $" + price);
    }
}
```

WebApp.java

```
public class WebApp implements Observer {
    public void update(double price) {
        System.out.println("Web App - Stock price: $" + price);
    }
}
```

StockMarket.java (Concrete Subject)

```
import java.util.*;

public class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private double price;

    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(price);
        }
    }
```

```
    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }
}
```

ObserverTest.java

```
public class ObserverTest {
    public static void main(String[] args) {
        StockMarket market = new StockMarket();

        MobileApp mobile = new MobileApp();
        WebApp web = new WebApp();

        market.registerObserver(mobile);
        market.registerObserver(web);

        market.setPrice(150.50);
        market.setPrice(155.75);
    }
}
```

Output:

```
Mobile App - Stock price: $150.5 Web App - Stock price: $150.5 Mobile App - Stock
price: $155.75 Web App - Stock price: $155.75
```

Exercise 8: Strategy Pattern

Scenario: Payment system with runtime payment method selection

PaymentStrategy.java (Strategy Interface)

```
public interface PaymentStrategy {
    void pay(double amount);
}
```

CreditCardPayment.java

```
public class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    public void pay(double amount) {
        System.out.println("Paid $" + amount +
            " via Credit Card: " + cardNumber);
    }
}
```

PayPalPayment.java

```
public class PayPalPayment implements PaymentStrategy {
    private String email;

    public PayPalPayment(String email) {
        this.email = email;
    }

    public void pay(double amount) {
        System.out.println("Paid $" + amount + " via PayPal");
    }
}
```

PaymentContext.java (Context)

```
public class PaymentContext {
    private PaymentStrategy strategy;

    public PaymentContext(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void executePayment(double amount) {
        strategy.pay(amount);
    }
}
```

StrategyTest.java

```
public class StrategyTest {
    public static void main(String[] args) {
        PaymentContext context;

        context = new PaymentContext(
            new CreditCardPayment("1234-5678-9012-3456"));
        context.executePayment(100.00);

        context = new PaymentContext(
            new PayPalPayment("user@example.com"));
        context.executePayment(75.50);
    }
}
```

Output:

```
Paid $100.0 via Credit Card: 1234-5678-9012-3456 Paid $75.5 via PayPal
```

Exercise 9: Command Pattern

Scenario: Home automation system with remote control commands

Command.java (Command Interface)

```
public interface Command {
    void execute();
}
```

Light.java (Receiver)

```
public class Light {
    public void turnOn() {
        System.out.println("Light is ON");
    }

    public void turnOff() {
        System.out.println("Light is OFF");
    }
}
```

LightOnCommand.java

```
public class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.turnOn();
    }
}
```

LightOffCommand.java

```
public class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.turnOff();
    }
}
```

RemoteControl.java (Invoker)

```
public class RemoteControl {
    private Command command;
```

```
    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }
}
```

CommandTest.java

```
public class CommandTest {
    public static void main(String[] args) {
        Light light = new Light();
        RemoteControl remote = new RemoteControl();

        remote.setCommand(new LightOnCommand(light));
        remote.pressButton();

        remote.setCommand(new LightOffCommand(light));
        remote.pressButton();
    }
}
```

Output:

```
Light is ON Light is OFF
```

Exercise 10: MVC Pattern

Scenario: Student record management application using MVC architecture

Student.java (Model)

```
public class Student {
    private int id;
    private String name;
    private String grade;

    public Student(int id, String name, String grade) {
        this.id = id;
        this.name = name;
        this.grade = grade;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getGrade() {
        return grade;
    }

    public void setGrade(String grade) {
        this.grade = grade;
    }
}
```

StudentView.java (View)

```
public class StudentView {
    public void displayStudentDetails(Student student) {
        System.out.println("Student ID: " + student.getId());
        System.out.println("Student Name: " + student.getName());
        System.out.println("Student Grade: " +
            student.getGrade());
    }
}
```

StudentController.java (Controller)

```
public class StudentController {
    private Student model;
```

```
    private StudentView view;

    public StudentController(Student model,
            StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void updateName(String name) {
        model.setName(name);
    }

    public void updateGrade(String grade) {
        model.setGrade(grade);
    }

    public void displayStudentDetails() {
        view.displayStudentDetails(model);
    }
}
```

MVCTest.java

```
public class MVCTest {
    public static void main(String[] args) {
        Student student = new Student(101, "John", "A");
        StudentView view = new StudentView();
        StudentController controller =
            new StudentController(student, view);

        System.out.println("Initial Student Data:");
        controller.displayStudentDetails();

        controller.updateName("John Doe");
        controller.updateGrade("A+");

        System.out.println();
        System.out.println("Updated Student Data:");
        controller.displayStudentDetails();
    }
}
```

Output:

```
Initial Student Data: Student ID: 101 Student Name: John Student Grade: A
Updated Student Data: Student ID: 101 Student Name: John Doe Student Grade: A+
```

Exercise 11: Dependency Injection

Scenario: Customer management with constructor-based dependency injection

Customer.java

```
public class Customer {
    private int id;
    private String name;

    public Customer(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

CustomerRepository.java (Interface)

```
public interface CustomerRepository {
    Customer findCustomerById(int id);
}
```

CustomerRepositoryImpl.java

```
public class CustomerRepositoryImpl
        implements CustomerRepository {

    public Customer findCustomerById(int id) {
        if (id == 1) {
            return new Customer(1, "Alice");
        } else if (id == 2) {
            return new Customer(2, "Bob");
        } else if (id == 3) {
            return new Customer(3, "Charlie");
        }
        return null;
    }
}
```

CustomerService.java

```
public class CustomerService {
    private CustomerRepository repository;

    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }
```

```
    public void findAndDisplayCustomer(int id) {
        Customer customer = repository.findCustomerById(id);
        if (customer != null) {
            System.out.println("Customer Found:");
            System.out.println("ID: " + customer.getId());
            System.out.println("Name: " + customer.getName());
        } else {
            System.out.println("Customer not found");
        }
    }
}
```

DependencyInjectionTest.java

```
public class DependencyInjectionTest {
    public static void main(String[] args) {
        CustomerRepository repo =
            new CustomerRepositoryImpl();
        CustomerService service =
            new CustomerService(repo);

        System.out.println("Finding Customer 1:");
        service.findAndDisplayCustomer(1);

        System.out.println();
        System.out.println("Finding Customer 2:");
        service.findAndDisplayCustomer(2);

        System.out.println();
        System.out.println("Finding Customer 3:");
        service.findAndDisplayCustomer(3);
    }
}
```

Output:

```
Finding Customer 1: Customer Found: ID: 1 Name: Alice  Finding Customer 2:
Customer Found: ID: 2 Name: Bob  Finding Customer 3: Customer Found: ID: 3 Name:
Charlie
```